Contextual Inference

Paper Structure — Results Summary

Cheshta Bhatia

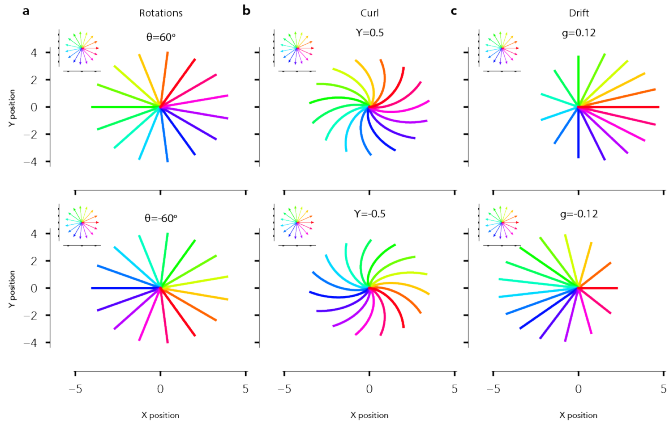
June 2026

1 MTL infers latent context from cerebellar prediction errors

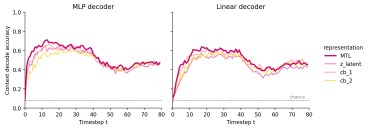
We model the MTL–cerebellar circuit such that MTL can infer context from the cerebellum's prediction error. A cerebellar-like forward model—a feedforward MLP (CB)—predicts the next state from the current state, action, and the MTL latent context as inputs, and the state prediction error $e_t = s_t - \hat{s}_t$ is fed back to MTL. A recurrent MTL-like module (an RNN) receives these prediction errors and infers the latent context driving them, giving a context code z_{MTL} as output that conditions the forward model. The whole circuit is trained only to predict the next state (MSE); the MTL module is never told the context—it must discover it from the prediction errors.

We train under a family of contexts – changing dynamics spanning three perturbation families—*rotation* of the action, velocity-dependent *curl*, and constant *drift*—each at four signed magnitudes, for twelve contexts (Fig. 1a).

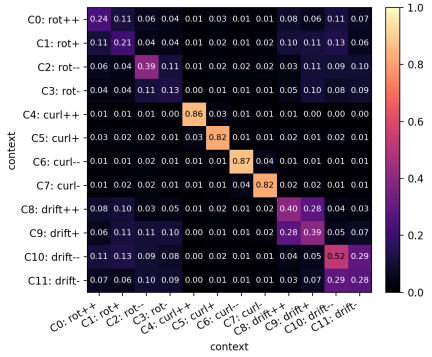
Context is decodable from z_{MTL} far above chance ($1/12 \approx 0.08$) by both linear and nonlinear decoders, and decodability builds up over the trial as the MTL module accumulates prediction errors (Fig. 1b). The confusion matrix reports both how reliably each context is identified and which contexts are mistaken for one another (off-diagonal). Confusions are confined within family—a context is mixed up only with members of its own family and never across families (a rotation is never taken for a drift). Decodability varies with family: curl contexts are cleanly separated, whereas rotation and drift magnitudes are more confusable with one another (Fig. 1c). The decoder's per-context weight directions reveal a clear organisation: contexts of the same family and sign are organised and clustered together, different families occupy distinct directions, and opposite signs within a family are represented separately (Fig. 1d). The inferred code thus mirrors the factors of the contextual dynamics—family, direction, and magnitude—rather than treating the twelve contexts as arbitrary labels.



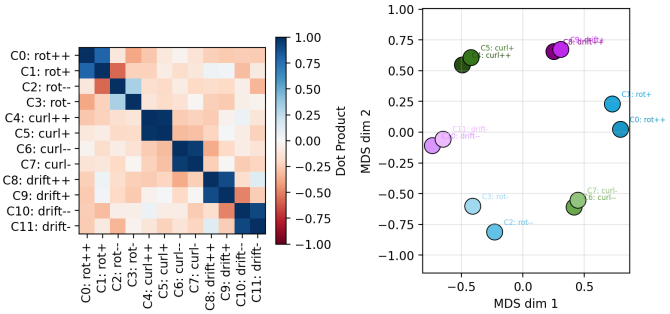
(a)



(b)



(c)



(d)

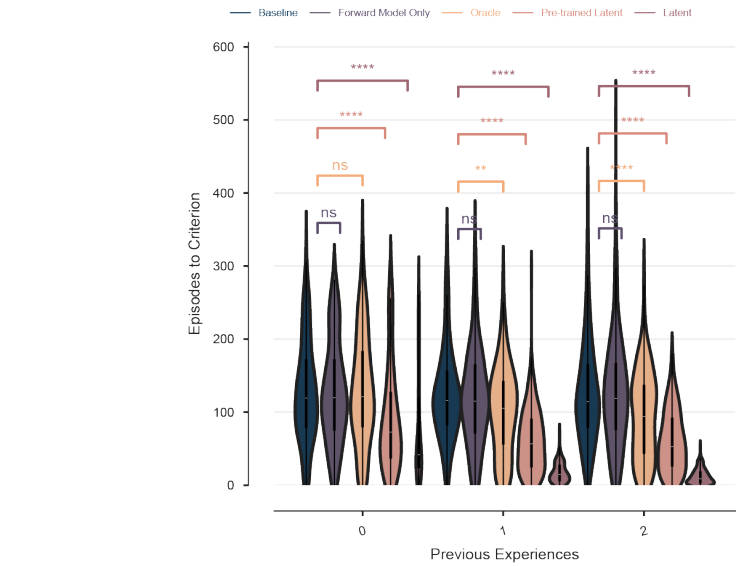
Figure 1: **MTL infers latent context from cerebellar prediction errors. (a)** Task. The MTL to CB network is trained only on next-state prediction (MSE) and is never given the context. Dynamics span three perturbation families—rotation, curl, and drift—each at four signed magnitudes (12 contexts). **(b)** Context decoded from z_{MTL} over the trial, far exceeding chance ($1/12 \approx 0.08$) under both linear and nonlinear (MLP) decoders; decodability builds up as the recurrent module accumulates prediction errors. **(c)** Symmetric, row-normalized confusion matrix of the context decoder. The diagonal gives per-context decodability; off-diagonal is confined to the same-family block (cross-family confusions ≈ 0). Curl contexts are cleanly separated, while rotation and drift magnitudes are more confusable among themselves. **(d)** Cosine similarity between the decoder's per-context weight directions (left) and its MDS embedding (right). Contexts of the same family and sign are read along a common direction and cluster together, different families occupy distinct directions, and opposite signs within a family are represented separately.

2 Inferred context produces savings, not mere error correction

We have shown that MTL infers latent context; we now ask whether this inference is behaviorally useful. We add a motor-cortex-like controller—a soft actor–critic (SAC) reinforcement learner—that performs a navigation task in a 4×4 grid: each episode the start and goal locations are randomly chosen, and the agent is rewarded for approaching the goal. The movement has the same dynamics as before, with each context—rotation, curl, or drift– applying a distinct perturbation. The MC module selects actions – its inputs include current state, goal, context latents supplied by the MTL–cerebellar circuit, z_{MTL} (from MTL) and z_{cb} (from the cerebellum).

Savings is the reuse of retained structure: a previously-seen context is relearned faster on its later encounters than on its first. To test contextual inference, We cycle the contexts $A \to B \to C \to D$, repeated three times, and measure *episodes-to-criterion*—the episodes needed to reach a fixed performance threshold—at each recurrence. Online error correction predicts a flat episodes-to-criterion across repetitions; savings predict a decrease with each re-exposure. We observe the decrease (Fig. 2a,b), establishing savings rather than motor adaptation.

These savings depend on the MTL's representation and its ability to decode context. Unlike error correction, which is indifferent to context identity, savings require the agent to recognise the recurring context. When the inferred context is ablated from the agent's input (Baseline), the same (state, action) pair under different contexts becomes indistinguishable, the across-exposure speedup disappears, and each encounter is relearned from scratch. A controller given the cerebellar latent but not the MTL latent (Forward Model Only) likewise fails to accrue savings, isolating MTL as the necessary component; restoring the MTL latent recovers savings that exceed those of an oracle where the context is told explicitly as a one-hot label (Oracle) — inferred contextual code has information that transfers across re-exposures where bare context identity does not (Fig. 2a,b). Thus the speedup reflects context-inferred memory reuse, not online adaptation — and MTL's inferred context is necessary for it.



(a)



(b)

Figure 2: **Inferred context produces savings, not mere error correction.** Episodes-to-criterion—the episodes needed to reach a fixed performance threshold—as a function of the number of previous exposures to a context ($0 \rightarrow 1 \rightarrow 2$), for five conditions. *Baseline*: SAC controller with state and goal only (no context). *Forward Model Only*: additionally given the cerebellar latent z_{cb} but not the MTL latent. *Latent*: given the MTL latent z_{MTL} from the MTL–cerebellar circuit. *Pre-trained Latent*: as Latent, with the circuit frozen at its pretrained (Sec. 1) weights. *Oracle*: given the true context as a one-hot code. **(a)**Pooled across contexts, episodes-to-criterion falls with re-exposure for the Latent and Pre-trained Latent conditions (savings), but stays flat for Baseline and Forward-Model-Only (no savings; the two are not significantly different), with the inferred Latent and Pre-trained Latent surpassing the one-hot Oracle. **(b)** The same, split by context (rot+, curl+, rot−, curl−): the savings pattern holds across contexts. Brackets: $^{****}p < 10^{-4}$, ns = not significant.

3 MTL learns the structure of a context family and generalizes to unseen contexts

Sections 1–2 show that MTL infers the latent context and that this inference is behaviorally useful. However, does MTL memorize the contexts it has experienced, or instead learns the structure of the family they are drawn from. If it learns the structure, it should be able to extend that knowledge to an unseen context from the same family and predict its dynamics zero-shot.

We test this on the MTL–cerebellar network, trained only on next-state prediction (MSE). We use two parametric context families: a *rotation* family, in which the perturbation rotates the action by angle θ ($F = R(\theta)\,a$), and a *drift* family, in which a constant force of magnitude d is added along $+x$ ($F = a + d\,[1, 0]$). The network is trained on three anchor values of each axis ($\theta \in \{0°, 45°, 90°\}$; $d \in \{0.4, 0.8, 1.2\}$) and then tested on a sweep of held-out values, both interpolating between the anchors and extrapolating beyond them.

Across held-out values within the trained range, the cerebellar prediction error remains low and smooth, with no spikes between the anchors (Fig. 3a,c, blue). The flat-low MSE curve shows the network has learned the underlying structure—rotation angle and drift magnitude as continuous variables—rather than a lookup table and is able to interpolate it to unseen contexts from the same family. Extrapolating beyond the trained range raises the error (Fig. 3a,c), so the structural manifold is learned where it is sampled and degrades outside it.

Two controls in the same panels isolate the source of this generalization. A context-conditioned cerebellum given a one-hot label of the trained context—but no metric structure—incurs higher error between the anchors (Fig. 3a, orange), confirming that smooth generalization requires the continuous code, not merely knowing the context identity. However, this result is confounded by the fact that z_{MTL} is 64-dimensional whereas a scalar context label is one number, causing a capacity mismatch. To rule it out, we add a matched oracle: an MLP is given the same state and action inputs the MTL circuit sees, but handed the true θ/d in place of the cerebellar prediction error, and it outputs its own learned 64-D code for the same cerebellum. The oracle and MTL thus feed the cerebellum the same size (64-D) input and differ only in told-versus-inferred context. MTL's inferred context yields slightly lower prediction error than the told-θ oracle (Fig. 3a,c, blue below purple). We suspect that is because the two receive different signals: the oracle is given the context θ and must learn its mapping onto the dynamics, whereas MTL reads the cerebellar prediction error, and leverages that to infer the perturbation dynamics as it interacts with every action and is therefore a more task-relevant code. This shows that inferred context is, if anything, a better basis for structure learning than explicit context information.

The learned structure is visible directly in the MTL representation. A linear decoder fit on z_{MTL} at the three trained anchors only recovers the held-out rotation angles, tracking the identity line within the trained range and saturating beyond it (Fig. 3b). Thus z_{MTL} encodes θ as a continuous axis: held-out

contexts are placed by interpolation on a learned manifold, not retrieved from memory. The drift family makes the same point at the level of prediction rather than decoding. Because a $+x$ drift is a simple translation, the network can interpolate to held-out drift magnitudes without representing d as a sharply decodable axis — so for drift the evidence is the smooth, low generalization-error curve (Fig. 3c), while the decode (Fig. 3d) is correspondingly weak.

A leave-one-out control shows that training on only the two endpoints ($\theta \in \{0°, 90°\}$) produces an error bump at the omitted midpoint near 45°, precisely where no training anchor was provided; the bump vanishes once 45° is included as a training anchor (Fig. 3e). The midpoint error is, however, smaller than the network's extrapolation error outside the trained range, indicating that the representation still interpolates within the gap—just less accurately than when the gap is itself used for training. Together, the rotation and drift families show that MTL recovers the generative structure of a context family and uses it to predict unseen contexts, rather than memorizing them.

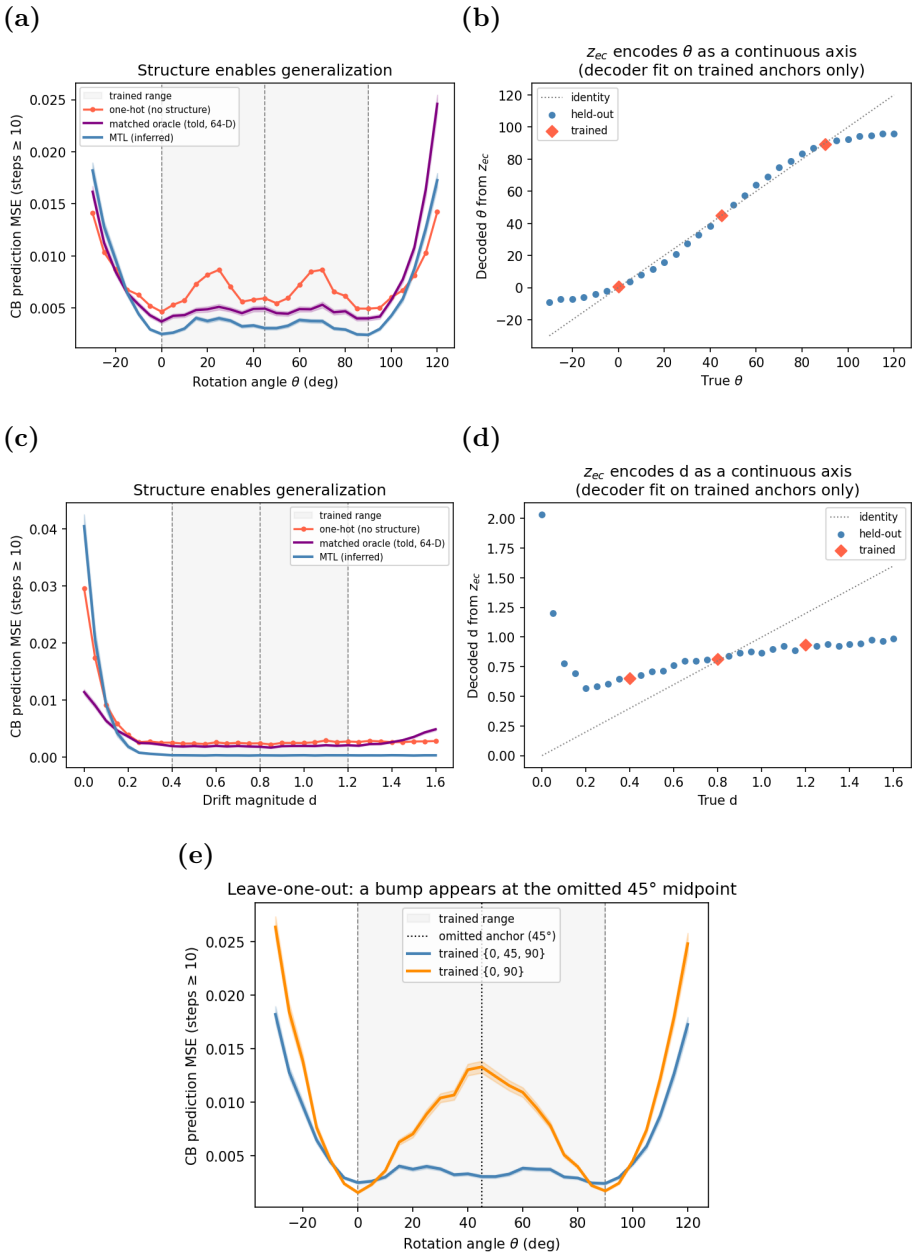


Figure 3: **MTL learns the structure of a context family and generalizes to unseen contexts.** The MTL–cerebellar forward model is trained on three anchor values of each axis ($\theta \in \{0°, 45°, 90°\}$; $d \in \{0.4, 0.8, 1.2\}$) and tested on held-out values. **(a)** Rotation: cerebellar prediction error vs. θ. MTL (blue) stays low and smooth across held-out angles within the trained range and rises on extrapolation; a one-hot label (orange) incurs higher error between anchors, while the capacity-matched oracle (purple) is matched—and at most points slightly exceeded—by MTL. **(b)** θ decoded from z_{ec} by a linear decoder fit on the trained anchors only: held-out angles track the identity line within range and saturate beyond it. **(c)** Drift: cerebellar prediction error vs. d, same conditions; the generalization curve carries the evidence for this translation-only family. **(d)** d decoded from z_{ec} by a linear decoder fit on trained anchors only. **(e)** Leave-one-out control: training on only the endpoints ($\theta \in \{0°, 90°\}$) produces an error bump at the omitted midpoint near 45° (orange) that is absent when 45° is a training anchor (blue).

4 Context inference is prediction-error agnostic: reward-prediction error enables contextual inference too

In Sections 1–3 the error signal MTL reads is a vector *state*-prediction error from the cerebellar forward model. We now ask whether context inference is tied to that particular signal, or whether MTL can infer context from a different prediction error too —a scalar *reward*-prediction error (RPE). We test this on the serial-reversal stimulus–response–outcome (SRO) task of Bernardi et al. (2020): the task has two uncued latent contexts and 4 stimuli images, where each (stimulus, context) has a correct operant (hold/release) and reward. The context switches every 50–70 trials and flips all four mappings at once, and is never cued. The agent must infer which context it is in to get reward.

The model is the same MTL–cerebellar circuit, with one change: the input signal to MTL is the scalar RPE $\delta_t = r_t^* - \hat{r}_t$, replacing the state-prediction error. MTL sends z_{MTL} to the cerebellar module and it outputs the operant and predicted reward, trained with $\mathrm{BCE}(\hat{O}, O^*) + \mathrm{MSE}(\hat{r}, r^*)$.

Context is decodable from z_{MTL} at 0.98 accuracy (chance 0.5). Context inference is evident behaviourally (Fig. 4b): on the first post-switch appearance of a stimulus the agent is below chance, because it is still used to the previous context and the reversal makes the old mapping wrong; but from the second appearance onward it is at ceiling—having observed a single post-switch outcome, it infers the new context and responds correctly to the other stimuli on their first post-switch encounter. This fast rise in context inference (cf. Bernardi Fig. 1C) is the hallmark of latent inference rather than gradual relearning. Thus MTL's context inference is a general, error-driven mechanism: it operates from a scalar reward-prediction error just as well as from a state-prediction error.

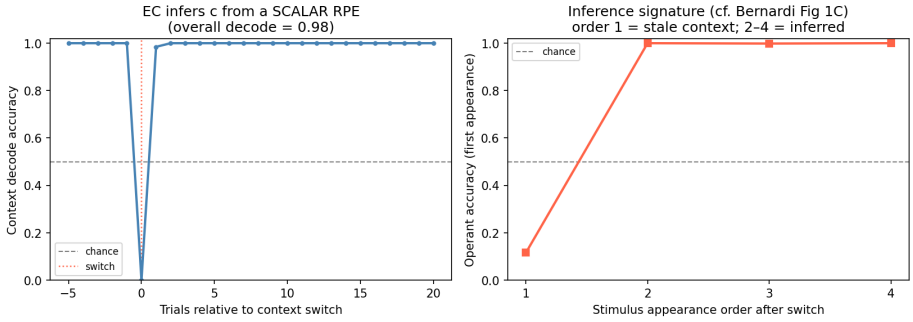


Figure 4: **Context inference from a scalar reward-prediction error (serial-reversal SRO task). (a)** Context decoded from z_{MTL}, aligned to the uncued context switch (overall decode 0.98, chance 0.5): decoding collapses for one trial at the switch and rises again within one to two trials. **(b)** Inference signature : operant accuracy on each stimulus's first post-switch appearance, by appearance order. The first stimulus is below chance, while appearances 2–4 are at ceiling—one observed outcome is enough to infer the new context

5 MTL representations match hippocampal recordings

Sections 1–4 show that MTL infers latent context. We now ask whether the geometry and representations of the inferred-context code in MTL hidden state resembles that of real hippocampal neural recordings. The comparison is at the level of representational properties(geometry/abstraction), not same-stimulus matching, since the tasks and stimuli differ.

5.1 Motor (non-SRO) task: comparison to ventral hippocampus

The motor model of Sections 1–3 infers context from cerebellar state-prediction errors. We compare its MTL hidden state to recordings from ventral hippocampus(vCA1) in mice performing a hidden-state inference task (Mishchanchuk et al., 2024), in which animals infer the latent context—the identity of the currently high-reward lever—from the history of probabilistic outcomes, and vCA1 represents that context in an abstract format. Although this task differs in kind from our motor dynamics context, both require the agents to perform the same underlying computation: inferring the latent context from behavioral observations.

Applying the analyses used on vCA1 to the MTL hidden state reveals the same representational signatures (Fig. 5). Projected onto the directions that best separate contexts, the MTL population carries clear context structure—the curl family separates, with rotation and drift partially overlapping in two-

dimensional view and fully separating in three dimensions (Fig. 6).

Both the context and its family are linearly decodable well above chance, and context decodability is higher once the model has settled into a context after a switch (Fig. 5B). The context code is abstract: a context-type decoder trained on large magnitude contexts (stronger rotation, curl and drift) generalises to held-out magnitudes (smaller magnitudes of rotation, curl and drift), performing well above a label-shuffle null and improving as more units are included (Fig. 5C)—the cross-condition generalisation that defines an abstract representation in vCA1. Finally, a substantial fraction of MTL units are tuned to context, their activity strongly modulated by context (Fig. 5D), paralleling the context-tuned cells of vCA1. Because the forward model has no reward or choice variables, we restrict the comparison to the context representation; within it, the MTL hidden state reproduces the abstract, context-selective code that vCA1 uses to support hidden-state inference.

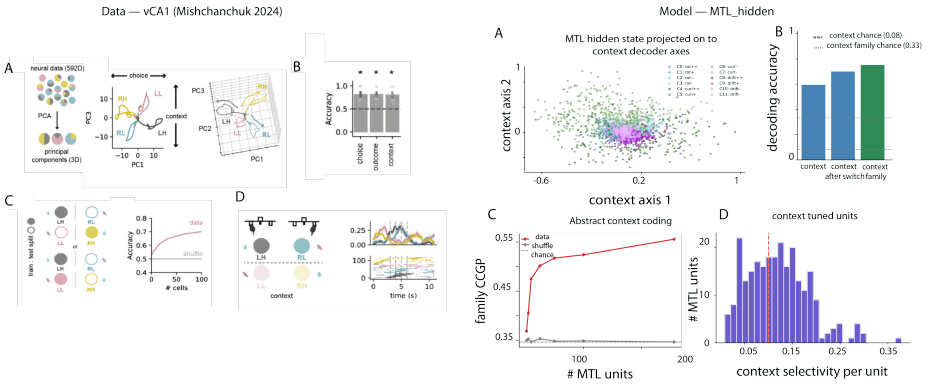


Figure 5: **Motor MTL hidden state codes for context like ventral hippocampus.** Left: vCA1 recordings during hidden-state inference (panels adapted from Mishchanchuk et al., 2024). Right: model MTL hidden state (EC_hidden) on the motor task. **(A)** Population geometry: (left) vCA1 activity projected onto its top principal components, with choice, outcome, and context separating along distinct axes; (right) MTL activity projected onto the top context-decoder axes (supervised directions that best separate contexts), with contexts separable. **(B)** Linear decoding: (left) choice, outcome, and context decodable above chance; (right) context and context-family decodable above chance (context chance 0.08, family chance 0.33). **(C)** Abstraction (CCGP): a decoder trained on one set of conditions and tested on held-out conditions performs above the shuffle null and improves with population size — (left) vCA1 context abstraction vs. number of cells; (right) family-across-magnitude CCGP vs. number of MTL units. **(D)** Context selectivity: (left) an example vCA1 context cell, firing similarly within a context across trial types; (right) the distribution of per-unit context selectivity (η^2) across MTL units, a substantial fraction of which are context-tuned.

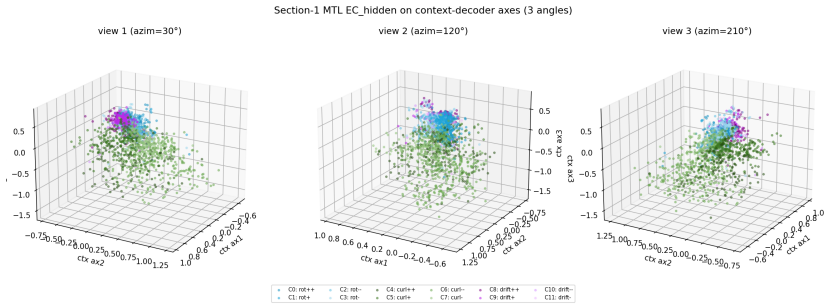


Figure 6: MTL hidden state on the top three context-decoder axes, three viewing angles. The curl family separates from rotation and drift, which overlap in 2D but resolve in 3D.

5.2 SRO task: comparison to hippocampal representations

The SRO model of Section 4 infers context from a scalar reward-prediction error in the serial-reversal task of Bernardi et al. (2020), in which monkeys infer the latent context—the current stimulus–response–reward mapping—-on the basis of the outcomes of their choices. After a reversal, a response that was previously rewarded for a given stimulus (holding rather than releasing the button) is no longer rewarded; animals use this to infer the context.

Simultaneous recordings from hippocampus (HPC), DLPFC, and ACC show that context is decodable and represented abstractly in HPC. The MTL network trained on the same task represents the inferred context in the same abstract format. Among all dichotomies, context is the one that generalises: a context decoder trained on some stimuli correctly reads out context on held-out stimuli it never saw, giving a high CCGP and parallelism score, whereas value and action do not generalise (Fig. 7A,B). Aligned to an uncued switch, context abstraction collapses briefly and then locks back in over the first few trials (Fig. 7C,D), reflecting ongoing inference. The MTL hidden state is high-dimensional: its shattering dimensionality lies well above the shuffle null, decoding most dichotomies (Fig. 7E), so the model matches HPC on both context abstraction and dimensionality. The MTL hidden state thus reproduces the hippocampal abstraction of the inferred context within a comparably high-dimensional geometry.

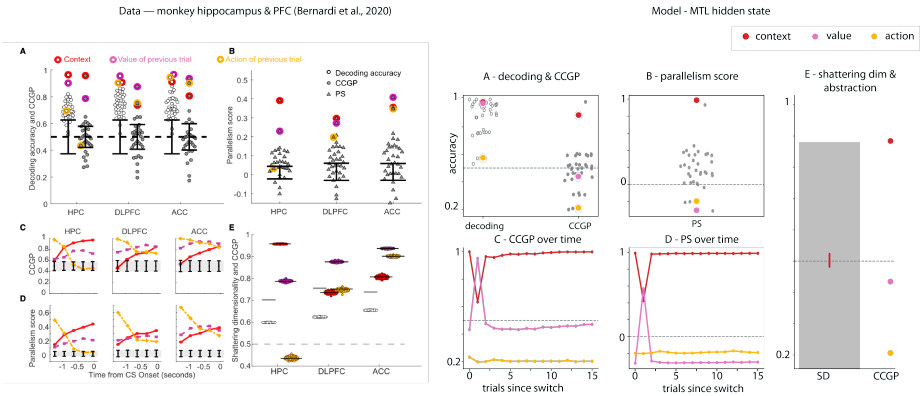


Figure 7: **SRO MTL hidden state codes for context abstractly, like the hippocampus, and is high-dimensional.** Left: HPC/DLPFC/ACC recordings on the serial-reversal task, pre-stimulus epoch (panels adapted from Bernardi et al., 2020; HPC is the comparison). Right: model MTL hidden state (h_t). In both, context is red, value pink, action yellow. **(A)** Decoding accuracy (open) and CCGP (filled) over the 35 dichotomies of the 8 conditions: context is abstract (high CCGP) in HPC and in the model, whereas value and action are not. **(B)** Parallelism score, confirming context abstraction. **(C,D)** CCGP and PS as a function of trials since a context switch; context abstraction builds up and locks in after the switch. **(E)** Shattering dimensionality (mean decoding over all dichotomies) versus a shuffle null: the recorded populations are abstract and high-dimensional, same for the model.

To Do

1. **Does structure-learning generalization occur with the learning component present?**

   The structure learning result in Section 3 currently rests on the forward prediction-only network (MTL–cerebellar). We observe that the prediction error stays low and z_{MTL} can generalise to held out angles that are in the learned manifold range. However, this only shows that the network can generalise context inference to unseen angles; what we still do not know is whether the agent can actually select appropriate actions for a never-seen context without relearning.

   Running the experiment with the motor cortex present therefore tests whether representational generalization translates into behavioral (zero-shot) generalization. To achieve this, we will repeat the experiment and train on anchors $\theta \in \{0°, 45°, 90°\}$, test on held-out angles; leave-one-out on the endpoints $\{0°, 90°\}$ to recover the mid-range error bump) with the full MTL–cerebellar–motor-cortex circuit, and confirm the smooth within-range generalization and the midpoint bump persist.

2. **Does the neural representation in MTL during the SRO task change, when performed by the full MTL-MC-CB circuit?**

   In the current SRO model, the cerebellar module both predicts reward and selects the operant, so action selection lives inside the inference circuit. If the action is produced by a dedicated MC rather than the cerebellum, does the MTL representational geometry change —does h_t become a more specific context code (since it offloads the already-non-abstract action representation to MC), and does the shattering dimensionality shift? To answer this, we will add an MC that selects the operant from z_{MTL}, retrain, and re-run the SRO geometry analysis (decoding / CCGP / parallelism / shattering dimensionality), and compare the MTL geometry with and without the dedicated motor module.

3. **Can the neural circuit also perform cross-task generalization?** Section 3 establishes generalization within a family — we already know that learning a few values of a perturbation allows the network to generalize to other values of the same task—training on a handful of rotation angles (or drift magnitudes) helps in interpolation to novel angles in the same family. Here we ask : does this generalization also happen across tasks? That is, has MTL learned the abstract logic of a task—structure that it could leverage and carry to stimuli it has never seen—rather than a set of memorized stimulus–response mappings. Demonstrating this would show the model has captured a task's generative structure and acquired the logic to solve it, which is what would let a single circuit solve a family of tasks rather than one. We test this in the SRO task by training on the existing SRO images and testing on novel set of images for which the SRO logic still holds (context flips the mapping) but which the agent has never encountered.